

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 6 Модули

Модули в Julia позволяют организовывать код в отдельные, независимые блоки, что упрощает управление большими проектами, улучшает структуру кода и способствует его повторному использованию.

6.1 Создание модулей

Модуль создаётся с помощью ключевого слова **module**, и завершается ключевым словом **end**.

```
In [4]: module NewModule
function hello(name)
    println("Привет, $name !")
end
end
```

```
Out[4]: Main.NewModule
```

В этом примере создаётся модуль с именем MyModule, в котором определена функция hello.

Чтобы использовать модуль, его нужно импортировать с помощью ключевого слова **using** или **import**.

```
In [5]: # если модуль в том же файле
using .NewModule
NewModule.hello("Юля")
```

Привет, Юля !

В случае, если нужный нам файл находится не там же, где и исполняемый файл Julia, необходимо будет создать файл с модулем, с расширением **.jl**, а так же указать путь к файлу следующим образом: **include("<путь к файлу><имя файла>.jl")**

Код в файле Hello_Julia.jl:

```
module Hello
function hello()
    println("Привет, мир !")
end
end
```

```
In [6]: include("/home/jusya/Hello_Julia.jl") # Подключаем модуль
using .Hello # Используем модуль
Hello.hello() # Вызов функции внутри модуля
```

Привет, мир !

Если вы хотите, чтобы какие-то функции или переменные из модуля были доступны напрямую, их нужно экспортировать с помощью ключевого слова **export**.

```
In [2]: module Add
export add
function add(a, b)
    return a + b
end
end

using .Add
add(2, 3)
```

Out[2]: 5

Переменные, объявленные внутри модуля, не доступны напрямую извне, если они не экспортируются.

```
In [3]: module Test1
export x1
x1 = 10
end

using .Test1
println(x1)
```

10

Если бы переменная не была экспортирована, доступ к ней был бы возможен только через полное имя модуля.

```
In [4]: module Test2
        x2 = 10
        end

        using .Test2
        println(Test2.x2)
```

10

6.2 Модули внутри других модулей

В Julia можно создавать модули внутри других модулей.

```
In [5]: module Test3
        module Test4
            export hello
            function hello(name)
                println("Привет из Test4, ", name)
            end
        end
    end

    using .Test3.Test4

    hello("Алекс")
```

Привет из Test4, Алекс

```
In [6]: module Test5

        module Test6
            function hello(name)
                println("Привет из Test6, ", name)
            end
        end

        function hello(name)
            println("Привет из Test5, ", name)
        end

        end

        using .Test5

        Test5.hello("Алиса")
        Test5.Test6.hello("Саша")
```

Привет из Test5, Алиса

Привет из Test6, Саша

6.3 Разрешение конфликтов имени

Если два модуля экспортируют одну и ту же функцию или переменную, можно использовать полное имя модуля для явного указания, какую именно функцию вы хотите использовать.

```
In [7]: module A
        export f
        function f()
            println("Функция из модуля A")
        end
    end

    module B
        export f
        function f()
            println("Функция из модуля B")
        end
    end

    using .A
    using .B

    # Обращение через полное имя
    A.f()
    B.f()
```

Функция из модуля A

Функция из модуля B

Так же можно переименовать функцию при импорте с помощью ключевого слова **as**. Это позволяет избежать путаницы и легко различать функции с одинаковыми именами.

```
In [8]: using .A
        using .B

        # Переименование функции из модуля A
        using .A: f as fA

        # Переименование функции из модуля B
        using .B: f as fB

        # Теперь можно использовать переименованные функции
        fA()
        fB()
```

Функция из модуля A

Функция из модуля B

6.4 Стандартные и пустые модули

В Julia есть **стандартные модули**, которые автоматически доступны, и **пустые модули**, которые можно создавать самостоятельно.

Стандартные модули, которые доступны всегда и не требуют загрузки:

- **Core** – содержит все функциональные возможности, встроенные в язык;
- **Base** – содержит базовый функционал;
- **Main** – это модуль, в котором выполняется код, когда вы запускаете программу или работаете в REPL.

В Julia модули по умолчанию включают **using Core** и **using Base**, но если эти стандартные определения не нужны, можно использовать `baremodule` – тогда модуль будет содержать только `using Core`. Такие модули имеют следующую структуру:

```
baremodule ModuleName
```

```
end
```

Если вы не хотите использовать даже Core, то структура такого модуля будет выглядеть следующим образом:

```
ModuleName = Module(:ModuleName, false, false)
```

Пример. Создадим модуль, в котором с помощью макроса `@eval` (подробное описание макросов можно найти в главе 7) определим функции для выполнения базовых арифметических операций: сложения, вычитания, умножения и деления.

```
In [9]: # Создаём модуль без использования Core и Base  
NoCoreArithmetic = Module(:NoCoreArithmetic, false, false)  
  
@eval NoCoreArithmetic begin  
    # Функция для сложения двух чисел  
    add(x, y) = $(+)(x, y)  
  
    # Функция для вычитания  
    subtract(x, y) = $(-)(x, y)  
  
    # Функция для умножения  
    multiply(x, y) = $(*)(x, y)  
  
    # Функция для деления  
    divide(x, y) = $(/)(x, y)  
end
```

```
Out[9]: divide (generic function with 1 method)
```

```
In [11]: # Подключаем модуль  
using .NoCoreArithmetic  
  
# Используем функции, определённые в модуле  
println("Сложение: ", NoCoreArithmetic.add(10, 5))  
println("Вычитание: ", NoCoreArithmetic.subtract(10, 5))
```

```
println("Умножение: ", NoCoreArithmetic.multiply(10, 5))  
println("Деление: ", NoCoreArithmetic.divide(10, 5))
```

Сложение: 15
Вычитание: 5
Умножение: 50
Деление: 2.0